Rapport på løsning Oblig 1 – IN2031

Lexer, parser og interpret for uttrykk

Matias Nupen matiasnu@ifi.uio.no

11. september 2025

1. Introduksjon

Denne rapporten dokumenterer løsningen på den første obligatoriske oppgaven i IN2031. Oppgaven er første del av fire, som samlet skal resultere i et domene-spesifikt-språk (DSL). Jeg bruker hver oppgave som et delmål i prosjektet, med GitHub som versjonskontroll og ene utviklingsbrancher for å strukturere arbeidet.

Målet med denne første delen er å implementere en enkel interpret som kan håndtere grunnleggende uttrykk. Rapporten redegjør for implementasjonen, hvordan jeg testet løsningen, og hvilke utfordringer og mulige forbedringer som ble identifisert.

2. Implementasjon

Programmet er strukturert rundt en ANTLR-bygd parser og lexer, en AST (Abstract Syntax Tree), og en interpreter som traverserer treet. Grammatikken ble gitt i oppgaven, mens jeg definerte uttrykkene basert på "Syntax of Expressions in AeroScript". Parseren bygger et tre fra input, og interpreter oversetter treet til resultater.

Programmet støtter tall (`NUMBER`), aritmetiske operasjoner (`PLUS, MINUS, TIMES`), unær negasjon (`NEG`), tilfeldige tall (`Random`, og `Random[x, y]`) samt punkter (`POINT(x, y)`). Alle uttrykk returnerer flyttall.

For operasjonene brukes en `OperationNode` med en `enum` som angir operasjonstype. Dette gir tydeligere feilmeldinger og er mer robust enn strengkonstanter. Jeg har også implementert en hjelpefunksjon `safeFloat()`, som logger og returnerer `NaN` dersom en operand er `null`, i stedet for å krasje programmet. Jeg har også implementert en jevnlig bruk av de definerte underreglene ANTLR gir, som for eksempel `ctx.point()` og `ctx.range()`, som gjør koden lettere å lese og mindre feilutsatt.

Jeg har også valgt å bruke flyttall som talltype i evalueringen. Testene som ble lagt ved har flyttall som utgangspunkt, og så tenker jeg at det også gir mening å bruke flyttall til å skaffe for eksempel

koordinatbruk. Eneste sted jeg bruker Integer er i Range, dette fordi det ønsker sjansen for ulike tall ved bruk av Random. Det er viktig å presisere at det derimot genereres et flyttall fra intervallet.

Mye av koden tok utgangspunkt i Calculator som ble vist i forelesning. Dette fordi implementasjonen er relativt lik, men justeringer tilpasset oppgaven er gjort.

Samlet sett gir denne implementasjonen en fungerende førsteversjon av DSL-en, med et klart skille mellom parsing, representasjon og evaluering. Dette legger et godt grunnlag for videre utvikling.

3. Testing

Jeg har testet implementasjonen både ved å kjøre de vedlagte automatiske testene og ved å bruke `println` for å skrive ut mellomresultater under utviklingen. De automatiske testene ble kjørt med `./gradlew test` og verifiserte at kjernefunksjonaliteten fungerte som forventet. `println` -metoden ble brukt i klasser som `OperationNode` for å se hvilke uttrykk som ble evaluert, og hvilke resultater som kom ut. Disse utskriftene ble senere fjernet, men var nyttige for å avdekke feil.

3.1. Utfordringer ved testing

En utfordring med denne tilnærmingen er at `println` -basert testing ikke verifiserer output automatisk, og dermed er bedre egnet til feilsøking enn til systematisk testing. Det gjør det vanskelig å fange opp regresjoner senere, siden man må tolke utskriftene manuelt. Automatiske tester gir mer pålitelig dekning, men i tilfelle med oppgaven dekker de i mindre grad feilmelding og uventede input.

4. Utfordringer og eventuelle problemer

En bevisst designbeslutning i oppgaven var å bruke flyttall som talltype. Dette gir fleksibilitet og gjør det enklere å håndtere uttrykk som `RANDOM` og `POINT`, hvor desimalverdier er naturlige. Samtidig kan flyttall introdusere utfordringer dersom DSL-en senere skal brukes til beregninger som krever eksakt nøyaktighet, ettersom avrundingsfeil kan oppstå. Videre kan bruken av `safeFloat()` som returnerer `NaN` i stedet for å stoppe programmet ved manglende operand, skjule logiske feil som burde vært håndtert tidligere. Til slutt er feilhåndteringen relativt enkel, og rapporterer i hovedsak generelle feil. Dette vil jeg argumentere er tilstrekkelig for denne oppgaven, men kan bli en begrensning når DSL-en utvides med mer kompleks funksjonalitet.

5. Konklusjon

Jeg har utviklet en førsteversjon av en DSL med støtte for tall, aritmetikk, negasjon, tilfeldige tall og punkter. Implementasjonen er strukturert og robust, og gir et godt grunnlag for videre utvidelser i de neste oppgavene.